

Q1-Please note that in this programming question, it is important that the naming of your classes, and signature of methods exactly follow the description. **All fields must be private and all methods must be public.**

This question is about implementing a program that can track and report the expenses and savings during months in a year. To this end, you need to implement two classes **Invoice** and **ExpensesTracker** as follows. We assume that the value of amount for invoices are already rounded numbers (whole numbers) and we assume that the arguments in method calls have valid values and you do not need to use defensive programming (check for validity of arguments). For instance you may assume that for a method that receives a month number the value of the argument is always between 1 and 12.

1-Implement a class called **Invoice**. The class has three fields **amount**, **month**, and **day** of type **int**. Furthermore, it has a field of type **String** called **payee**. Initialise all fields in the constructor with the value of the constructor parameters.

2-In class **Invoice**, implement a method **printInvoice()** which prints a string that contains the invoice information

"amount:<amount>, month:<month>, day:<day>, payee:<payee>"

for example for an invoice where amount= 200, day=31, month=9, payee="AB" it prints:  
amount:200, month:9, day:31, payee:AB

3-In class **Invoice**, implement methods **getAmount()**, **getMonth()**, **getDay()**, and **getPayee()** which return values of amount, month, day, and payee respectively.

4-Implement a class **ExpensesTracker** with a field called **income** of type **int** which represents the rounded value of monthly income. The class has another field of type **List<Invoice>** called **yearlyInvoices**. This list is used for keeping track of the invoices.

5-In the constructor of **ExpensesTracker** class, **yearlyInvoices** is initialised as an empty list and **income** is initialised with the only parameter of the constructor.

6-In the **ExpensesTracker** class, implement a method called **addInvoice(i)** which receives a parameter of type **Invoice** (e.g., i) and adds it to **yearlyInvoices**.

7-In the **ExpensesTracker** class, implement a method **monthExpenses(m)** which receives a parameter of type **int**. This method returns a whole number that is the total amount (sum of amounts) of invoices in month **m**. (**m** is the month number)

8-In the **ExpensesTracker** class, implement a method **monthSaving(m)** which receives a month number, that is a variable of type **int**. This method returns a whole number that is the total amount that has been saved in month **m** (by comparing the income and total amount of invoices in that month.)

9-In the **ExpensesTracker** class, implement a method **readInvoice()** that reads the information of an invoice from user input and adds the invoice to the **yearlyInvoices**. In the user input values of an invoice are given per line with the following order:

day  
month  
amount  
Payee

10-In the **ExpensesTracker** class, implement a method **clearMonthInvoices(m)** which receives a month number that is of type **int**. This method removes all the invoices in month **m**.

11-In the **ExpensesTracker** class, implement a method **listYearlySavings()** which prints the saved total amount in each month per line. If there was zero or less saving it prints "no saving in month <m>" where <m> is the month number, otherwise it prints "the saved amount in month <m> is <x>" where <x> is the amount that has been saved.

12-In the **ExpensesTracker** class, implement a method **payeeAmount()** that returns a map of type **Map<String,Integer>** that contains keys showing payees that appear in invoices in **yearlyInvoices** and the total amount that is paid to them.

Q2-Which of the following class declarations are valid, assuming classes **A** and **B** and interfaces **I** and **J** exist.

- a) public class C extends A {...}
- b) public class C implements I {...}
- c) public class C extends A implements I {...}
- d) public class C extends A, B implements I {...}
- e) public class C extends A implements I, J {...}
- f) public class C extends A, B implements I, J {...}

Q3-Consider **String[] str="par by par".split(" ");**

which of the following statement(s) evaluates to **true**?

- a) **str[0]==str[2];**
- b) **str[0].equals(str[2]);**
- c) **str[0]=="par";**
- d) **str[0].equals(str[1]);**

Q4-Consider the statement

```
Map m = new HashMap<>();
```

What is the dynamic and the static type of **m** respectively.

- a) Map and Map
- b) Map and HashMap
- c) HashMap and Map
- d) HashMap and HashMap

Q5-Consider the code below. Select correct statement(s) in the following?

```
public class Lab{...}
```

```
public class Main {
```

```
    public static void main ( String [] args ){
```

```
        var lab=new Lab("Robotics", 10);
```

```
        Lab.init();  
        lab.register(4);  
    }  
  
}
```

- a) **register** is a method defined in class **Main**
- b) in the code, **Lab** is a variable in method **main**
- c) **init** is a static method
- d) **Lab** has a constructor that takes two parameters

Q6-Select the correct statement(s) in the following.

- a) Only one constructor of a class is called when an object is created
- b) A local variable is only accessible during the execution of the block where it is declared
- c) A field should be declared inside a method
- d) A class must have multiple constructors

Q7-Assume that **b** is a variable of type **int**. Consider the following code:

```
int a=b%3;
```

Which of the following option(s) show a valid value for **a**?

- a) 0
- b) 3
- c) 1
- d) 4

Q8-Consider the code below. Which of the options show(s) a valid constructor header for the class?

```
public class MyClass {
```

```

private String name;
...
}

```

- a) public void MyClass(String name){this.name=name;}
- b) public MyClass(){name="J";}
- c) public String MyClass(){return name;}
- d) public MyClass(String name){this.name=name;}

Q9-Consider the following method

```

public static void foo(Map m) {
    m.put("Adam", 27);
    m.put("Bertil", 32);
    m.put("Cesar", 40);
    m.put("Bertil", 30);
    m.remove("Adam");
    m.put("Adam", 25); }

```

What are the contents of a map **m** after calling **foo(m)** assuming that **m** is empty before the call.

- a) "Adam" -> 27
- b) "Bertil" -> 32
- c) "Cesar" -> 40
- d) "Bertil" -> 30
- e) "Adam" -> 25

Q10-Consider the code below. Which option shows creating an object of myClass?

```

public class MyClass {
    private static int count;
    private Student student;

    public MyClass(int c){count=c;}

    public MyClass(Student student) {
        this.student=student;
    }
    ...
}

public class Student{...}

```

- a) var m=MyClass("Jack");
- b) var m=new MyClass(5, "Jack");
- c) var m=MyClass();
- d) var m=new MyClass(5);

Q11-Consider the following code. Which option(s) show correct replacements for (1) and (2) in the code, respectively? (assume class Item is defined.)

```
public class A{
    private Set<Item> items;

    public (1) check(Item item) {
        for((2) i: items){
            if(i.equals(item))
                return true;
        }
        return false;
    }
}
```

- a) boolean, Set<Item>
- b) boolean, Item
- c) Item, Item
- d) Set<Item>, boolean

Q12-Consider the following code

```
public class A {
    public void display() { System.out.println("Hello"); }
}

public class B extends A { }

public class C extends B {
    public void display() { System.out.println("World"); }
}
```

Which method(s) is called by the expression new B().display();

- a) The **display** method in class **Object**
- b) The **display** method in class **A**

- c) The **display** method in class **B**
- d) The **display** method in class **C**

Q13-What is coupling?

- a) It's a measure of how tightly interconnected classes are
- b) It's a measure of how many things a method/class/package are responsible for.
- c) It's a measure of how many keys in a map are coupled to a certain value.
- d) It's a measure of how many instances of a class are created during program execution.

Q14-Consider the following code

```
<code>s = new <code>();  
s.add("E"); s.add("D"); s.add("C"); s.add("B"); s.add("A");  
for(String str : s) { System.out.print(str);}
```

Mark replacement(s) of **<code>** that will guarantee that the output **ABCDE** is printed

- a) HashSet <String>
- b) TreeSet<String>
- c) LinkedList<String>
- d) ArrayList<String>

Q15-Assuming that the runtime exception **MyRuntimeException** and the checked exception **MyCheckedException** exist, which of the following method declarations are valid?

- a) public void foo() {  
    throw new MyRuntimeException();  
}
- b) public void foo() {  
    throw new MyCheckedException();  
}
- c) public void bar() throws MyCheckedException {  
    throw new MyCheckedException();  
}
- d) public void foo(boolean b) throws MyRuntimeException{  
    if (b) { throw new MyCheckedException(); }  
    else { throw new MyRuntimeException(); }  
}
- e) public void foo(boolean b) throws MyCheckedException {  
    if (b) { throw new MyCheckedException(); }  
    else { throw new MyRuntimeException(); }  
}

Q16-Consider the code below. Assume **reset** is called and successfully runs. Select the correct statement(s) at the end of method execution.

```
public class Main{  
    private int[] nums={10,20,30,40};
```

```

public void reset(int[] nums){
    this.nums=nums;
    nums[nums.length-1]=11;
}
...
}

```

- a) The value of **nums[nums.length-1]** changes to **11** in the class field
- b) Method parameter **nums** remain unchanged during the execution
- c) **nums[3]** is 40 in the field **nums** after running the method
- d) **length** for the class field **nums** is always 4 after running the method

Q17-Which of the following option(s) can be used for testing myMethod? (Assume the implementation shows the intended functionality)

```

public class A{

    public static boolean myMethod(boolean a, boolean b){
        return a||!b;
    }
}

```

- a) assertTrue(true, A.myMethod(true, true));
- b) assertTrue(A.myMethod(true, true));
- c) assertEquals(false, A.myMethod(true, true));
- d) assertEquals(true, A.myMethod(false, false));

Q18-Which statement(s) is/are true?

- a) You must always be able to use the object of a sub class whenever an object of a super class is expected.
- b) You must always be able to use the object of a super class whenever an object of a sub class is expected
- c) You may always use one object in place of another
- d) You may never use one object in place of another